

Class 4: Python — Control Flow and Data Structures

Data-driven Systems Engineering

Agenda

- Conditionals: making decisions with `if` / `elif` / `else`.
- For loops, `enumerate`, and list comprehensions.
- While loops and when to prefer them.
- Dictionaries: key–value pairs beyond the list.
- Writing your first functions with `def`.
- Why loops eventually hand off to NumPy and Matplotlib.

Learning goals

- Write conditional logic using comparison and boolean operators.
- Iterate over a list with a `for` loop, with and without `enumerate`.
- Build a list with a comprehension, with and without a filter condition.
- Create, read, and iterate over a dictionary.
- Define a small function with `def` and call it correctly.

Course Map and Running Cases



Today in the course

Focus: Control flow and the two data structures — lists and dictionaries — that carry most Python logic.

Bridge: Last class gave us values; this class gives those values a way to branch, repeat, and be looked up by name.

Fraud case

The fraud dashboard loops over 28 behavioral features to build one input vector, and a hyperparameter search is itself just a dictionary of lists.

Pasteurization case

The pasteurization simulator loops over sensor batches and branches on process state — “heating,” “holding,” “cooling” — using exactly this vocabulary.

Conditionals

- `if`, `elif`, and `else` branch execution based on a boolean expression.
- Comparisons: `==`, `!=`, `<`, `<=`, `>`, `>=`.
- Python uses indentation, not braces, to mark which statements belong to a branch.
- A colon `:` always introduces the indented block that follows.

Conditionals in practice

```
if prediction == 1:
    st.error(f"High risk of fraud: {probability:.2%}")
else:
    st.success(f"Transaction appears legitimate: {probability:.2%}")
```

- This is the decision at the heart of 'App_Credit_Card.py': one comparison, two branches.
- Note the # comment convention is skipped here because the branch itself is self-explanatory — a first taste of the readability habits we formalize in Class 7.

For loops

```
for i in range(1, 29):  
    col = cols[(i - 1) % 3]  
    user_inputs.append(  
        col.slider(f"Feature V{i}", -5.0, 5.0, 0.0, step=0.1)  
    )
```

- `for` iterates over a sequence — here, `range(1, 29)` produces the numbers 1 through 28.
- This is the real loop in 'App_Credit_Card.py' that builds the 28 feature sliders V1 through V28.
- The list `user_inputs` from last class is exactly what this loop fills in, one `.append()` at a time.

enumerate and list comprehensions

- `enumerate(sequence)` gives both the index and the value in one loop, avoiding manual counters.
- A list comprehension builds a list in one line: `[expr for item in sequence]`.
- Comprehensions can filter: `[expr for item in sequence if condition]`.
- Comprehensions do not replace every loop — they shine when the loop body is a single transformation.

Comprehension example

```
feature_names = [f"V{i}" for i in range(1, 29)]  
high_risk_features = [f for f in feature_names if f != "V1"]
```

- The first line generates the same 28 feature labels the sliders use, without a manual loop.
- The second line adds a filter condition — the comprehension form of an `if` inside a `for`.

While loops

- `while condition`: repeats a block as long as the condition stays true.
- Useful when the number of iterations is not known in advance, unlike a `for` loop over a fixed sequence.
- Streaming and monitoring code — reading sensor batches until a stream ends — is a natural `while` use case, previewed later in the pasteurization service.
- A missing update to the loop variable is the classic beginner bug: an infinite loop.

Dictionaries

```
param_grid = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [3, 5, 8, None],  
    'min_samples_split': [2, 4, 6]  
}  
  
for key, value in param_grid.items():  
    print(key, value)
```

- A dictionary stores key–value pairs, similar to a map: look up a value by its key instead of by position.
- This exact dictionary comes from 'notebooks/Class6_Model_Training_.ipynb'; note the values are themselves lists.
- `.items()` is the built-in way to iterate over both keys and values together.

Writing your first functions

- Functions are defined with `def name(parameters):` followed by an indented body.
- `return` sends a value back to the caller; without it, a function returns `None`.
- Wrapping repeated logic in a function is the first step away from copy-pasted code.
- Class 7 comes back to functions in depth — naming, structure, and good practice; today is only the syntax.

A first function

```
def summarize(values):  
    return min(values), max(values), sum(values) / len(values)  
  
low, high, avg = summarize(user_inputs)
```

- One function, three values returned as a tuple, unpacked directly into three variables.
- This is close to the practice exercise waiting in Class 5: mean, standard deviation, minimum, and maximum of a list, behind a Streamlit front end.

Why loops eventually hand off to NumPy

- A Python `for` loop over thousands of sensor readings or transaction rows is correct but slow.
- NumPy performs the same arithmetic on whole arrays at once, in compiled code under the hood.
- Matplotlib then turns those arrays into the plots we will build starting in Class 6.
- The mental model does not change: control flow, values, containers — only the execution engine gets faster.

How the repo already uses today's building blocks

Repo activity

- 'streamlit/pages/App_Credit_Card.py' uses exactly the `if/else` and `for` patterns shown today to build its inputs and its verdict.
- 'notebooks/Class6_Model_Training_.ipynb' defines `param_grid` as a dictionary of lists before any model is trained.
- Trace one loop and one dictionary in either file and describe, in your own words, what each iteration or lookup does.

Question

The slider loop in 'App_Credit_Card.py' runs exactly 28 times, always. Where in this course might a loop instead need to run an unknown number of times, and which loop would you reach for?

- Connects `for` versus `while` to a concrete forthcoming case: streaming sensor batches.

Papers and materials

[pandas documentation](#); [scikit-learn documentation](#); Chip Huyen, *Designing Machine Learning Systems*.

From This Class to the Next

What stays with us

- Conditionals and loops turn static values into behavior; dictionaries add lookup by name instead of position.
- Functions package repeated logic under one name, ready for reuse.
- The same handful of constructs already runs the fraud dashboard and the model-training notebook.

Project use

Before your project grows past a few lines, check whether a repeated block should already be a loop, a comprehension, or a function.

Next connection

Class 5 puts these constructs to work in a hands-on Streamlit practice session, building a small interactive app end to end.

Course thread: Vocabulary becomes skill only once you use it to build something that runs.